

Ch.6 Kubernetes 운영하기

목요일 오후, 회의실 한쪽에서 팀장이 화이트보드 앞에 서 있었습니다. 챕터 5에서 IP가 흔들려도 끊기지 않는 진입점을 만들어 둔 며칠이 지난 참이었습니다. 보드 위에는 챕터 3에서 docker-compose로 짰던 통합 사이트의 골격이 그대로 그려져 있었습니다. 프론트엔드와 백엔드, 그리고 그 뒤의 DB 한 덩어리.

팀장 : "구성 잘 잡았네요. 그럼 챕터 3에서 docker-compose로 짰던 그 통합 사이트, 이번엔 쿠버네티스 위에 올려 볼래요?"

오픈이는 펜을 든 채 잠시 멈춰있었습니다. Pod 하나, Service 하나는 깔끔하게 띄울 수 있습니다. 그런데 프론트엔드·백엔드·DB가 한 덩어리로 묶인 사이트를 그대로 옮기려고 하니, 어디부터 손대야 할지 막막했습니다. docker-compose로 띄울 때 짚지 않고 지나갔던 두 가지가 한꺼번에 도드라졌습니다.

먼저 DB 비밀번호가 마음에 걸렸습니다. 챕터 3에서는 `docker-compose.yml`에 환경 변수로 그냥 적어 뒀습니다. 로컬에서만 돌리는 것이라 그래도 됐습니다. 운영 환경 YAML에 같은 식으로 적으면 비밀번호가 Git 저장소에 그대로 올라갑니다. DB 주소나 접속 URL처럼 환경마다 달라지는 값도 마찬가지입니다. 그렇다고 이미지에 박아 두자니 환경이 바뀔 때마다 이미지를 다시 빌드해야 합니다.

데이터 쪽은 한 단계 더 무거운 문제였습니다. 챕터 4에서 Deployment가 죽은 Pod를 곧장 살려 주는 모습을 봤기 때문에, 같은 일이 DB Pod에서 벌어지면 어떻게 되는지가 또렷이 보였습니다. 코드는 다시 띄우면 그만입니다. 그 안에 쌓아 둔 회원 정보와 게시글은 새 Pod로 따라오지 않습니다.

회의실에서 자리로 돌아온 오픈이는 펜으로 노트에 두 단어를 적어 두었습니다. **설정·비밀번호**, 그리고 **데이터**. 두 매듭이 풀려야 챕터 3의 통합 사이트를 K8s 위로 옮길 수 있겠다는 그림이 머리에 그려졌습니다.

'두 매듭만 풀면 되는 거구나.'

오픈이의 한숨 소리에, 옆자리 선배가 의자를 굴려 다가왔습니다.

6.1 ConfigMap·Secret - 설정과 비밀번호를 이미지 바깥으로

6.1.1 이미지와 설정의 분리

선배 : "Dockerfile에 비밀번호를 직접 적지 마세요. 설정은 이미지 바깥에 두고, 이미지에는 순수한 코드만 담아야 해요."

오픈이는 화면을 다시 들여다봤습니다.

'바깥이라면 어디에 두어야 하지?'

설정값과 비밀번호를 따로 보관할 전용 공간이 필요했습니다. 쿠버네티스에는 이미 그 자리가 마련되어 있었습니다. 가맹점 한쪽에 공용 메뉴판을 걸어 두고, 비밀 레시피는 같은 가맹점 안에 따로 두는 방식입니다. 메뉴를 바꿀 때마다 주방까지 새로 들일 필요는 없습니다. 메뉴판과 레시피만 갈아 끼우면 됩니다. 쿠버네티스에서도 같은 발상으로 두 자리가 나뉘어 있습니다.

이 역할을 맡는 리소스는 두 가지입니다.

- **ConfigMap**은 일반적인 설정값을 저장할 때 사용합니다. DB 주소, 접속 URL, 로그 레벨처럼 환경에 따라 변하는 값을 담습니다.
- **Secret**은 비밀번호, 토큰, API 키처럼 민감한 정보를 저장할 때 사용합니다. ConfigMap과 구조는 비슷하지만, 값이 Base64로 인코딩되어 저장되며 접근 권한이 엄격하게 제한됩니다.



그림 6-1. ConfigMap과 Secret이 이미지와 별개로 Pod에 설정과 민감 정보를 주입

ConfigMap은 환경에 따라 달라지는 일반 설정값을 키-값 쌍으로 저장하는 리소스입니다. Pod에 환경 변수나 파일로 주입하여 이미지 재빌드 없이 설정을 바꿀 수 있습니다.

Secret은 비밀번호·토큰·키처럼 민감한 정보를 별도로 분리해 저장하는 리소스입니다. 값은 Base64로 인코딩되어 저장되며, RBAC으로 조회 권한을 제한할 수 있습니다.

6.1.2 ConfigMap

전체 실습 코드는 깃헙을 참고합니다.

실습 코드 (GitHub): <https://github.com/metacoding-10-linux-docker/docker/tree/master/ex13>

오픈이는 먼저 일반 설정을 관리하는 ConfigMap부터 손에 잡았습니다. ex13 폴더를 열어 키-값 두 개가 적힌 ConfigMap YAML을 펼쳤습니다.

ex13/configmap-conn.yml

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: configmap-conn          # ConfigMap 이름 지정
data:                          # 설정값 넣는 영역
  conn_info: "localhost:80"
  conn_url: "config.test"
```

data 영역에 키-값 쌍을 적는 것만으로 준비는 끝났습니다. 이어서 이 ConfigMap을 Pod가 가져다 쓰도록 Deployment에서 연결해 줍니다. `envFrom.configMapRef`를 쓰면 ConfigMap의 모든 값을 한꺼번에 환경 변수로 주입할 수 있습니다.

ex13/deploy-ex03.yml (핵심)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-config-secret      # Deployment 이름 (재시작 시 이 이름으로 지목)
spec:
  template:
    spec:
      containers:
        - name: nginx-container
          image: nginx:1.20
          envFrom:
            - configMapRef:
                name: configmap-conn  # ConfigMap 연결
```

YAML을 다시 들여다보면 `envFrom` 아래에 ConfigMap 이름 한 줄만 적혀 있습니다. 키를 하나씩 매핑해 줄 필요가 없습니다. 컨테이너가 시작되는 순간 `configmap-conn`에 적힌 `conn_info`와 `conn_url`이 그대로 환경 변수로 들어옵니다.

'이러면 애플리케이션 쪽 코드는 손댈 게 없네. 평소처럼 환경 변수만 읽으면 끝이잖아.'

오픈이는 두 YAML을 차례로 적용하고 Pod 안에 들어가 환경 변수를 찍어 봤습니다.

```
kubectl apply -f ex13/configmap-conn.yml
kubectl apply -f ex13/deploy-ex03.yml
kubectl exec -it <Pod명> -- env      # Pod 환경 변수 조회
```

실행 결과

```
$ kubectl get pod
NAME READY STATUS RESTARTS AGE
nginx-config-secret-794499d5d4-c2xmw 1/1 Running 0 11s
$ kubectl exec -it nginx-config-secret-794499d5d4-c2xmw -- env
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/bin
HOSTNAME=nginx-config-secret-794499d5d4-c2xmw
conn_info=localhost:80
conn_url=config.test
KUBERNETES_PORT_443_TCP_PORT=443
KUBERNETES_PORT_443_TCP_ADDR=10.96.0.1
KUBERNETES_SERVICE_HOST=10.96.0.1
KUBERNETES_SERVICE_PORT=443
KUBERNETES_PORT_443_TCP=tcp://10.96.0.1:443
KUBERNETES_PORT=tcp://10.96.0.1:443
```

그림 6-2. Pod 안의 환경 변수 목록에 ConfigMap의 값이 보이는 모습

Pod 내부의 환경 변수를 확인하니 `conn_info=localhost:80`처럼 ConfigMap에 적어 둔 값이 그대로 들어와 있었습니다. 이미지를 건드리지 않고도 설정값만 바깥에서 주입하는 데 성공했습니다.

6.1.3 Secret

전체 실습 코드는 깃헙을 참고합니다.

실습 코드 (GitHub): <https://github.com/metacoding-10-linux-docker/docker/tree/master/ex13>

남은 건 비밀번호였습니다. 같은 방식으로 ConfigMap에 비밀번호를 적어 두면 그만일까 생각이 들었지만, 곧 고개를 저었습니다. 민감한 값을 메뉴판처럼 공개된 곳에 적어 둘 수는 없습니다. 비밀번호는 가맹점 안 비밀 레시피처럼 따로 보관해야 할 값이고, 쿠버네티스에서는 그 자리를 Secret이라는 이름으로 따로 두고 있습니다.

ex13/secret-password.yml

```
apiVersion: v1
kind: Secret
metadata:
  name: secret-password
stringData:
  password: metacoding1234
```

`stringData` 를 사용하면 쿠버네티스가 내부적으로 값을 Base64로 인코딩하여 저장합니다.

```
kubectl apply -f ex13/secret-password.yml      # Secret YAML 적용
kubectl get secret secret-password -o yaml     # 저장된 Secret 내용을 YAML로 조회
```

실행 결과

```
$ kubectl get secret secret-password -o yaml
apiVersion: v1
data:
  password: bWV0YWNvZGluZzEyMzQ=
kind: Secret
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"v1","kind":"Secret","metadata":{"annotations":{},"name":"secret-
password"},"namespace":"default...
  creationTimestamp: "2026-03-15T06:30:27Z"
  name: secret-password
  namespace: default
  resourceVersion: "1387"
```

그림 6-3. Secret 내부를 보면 비밀번호가 Base64로 인코딩된 상태

저장된 결과를 보니 `password` 값이 알 수 없는 문자열로 변환되어 있습니다.

Secret과 Base64

Secret의 Base64 처리는 암호화가 아닌 단순 인코딩입니다. 따라서 누구나 디코딩하여 원문을 확인할 수 있습니다. 여기서는 '일반 설정과 민감 정보를 구분하여 관리한다'는 개념이 중요합니다.

Pod에 주입하는 방식은 ConfigMap과 동일합니다. 앞서 작성해 둔 `deploy-ex03.yml` 을 다시 열어 `envFrom` 아래의 `secretRef` 두 줄에 걸린 `#` 주석을 풀어 줍니다. `secretRef`를 활성화하면 쿠버네티스가 실행 시점에 Base64 인코딩을 디코딩해 환경 변수로 넣어 줍니다.

ex13/deploy-ex03.yml (Secret 연결 활성화)

```
envFrom:
  - configMapRef:
      name: configmap-conn
  - secretRef:
      name: secret-password # Secret 연결 (활성화)
```

```
kubectl apply -f ex13/deploy-ex03.yml # Secret 연결판 적용
```

실행 결과

```
$ kubectl get pod
NAME READY STATUS RESTARTS AGE
nginx-config-secret-7fbccb65f5-zq8nz 1/1 Running 0 51s
$ kubectl exec -it nginx-config-secret-7fbccb65f5-zq8nz -- env
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/bin
HOSTNAME=nginx-config-secret-7fbccb65f5-zq8nz
TERM=xterm
conn_url=config.test
password=metacoding1234
conn_info=localhost:80
KUBERNETES_SERVICE_PORT_HTTPS=443
KUBERNETES_PORT=tcp://10.96.0.1:443
```

그림 6-4. 환경 변수 목록에 Secret의 값이 평문으로 들어와 있는 모습

6.1.4 환경 변수 반영을 위한 Pod 재시작

여기까지 잘 돌아간다 싶었던 차에, 오픈이는 ConfigMap이 정말 살아 있는 설정인지 한 번 흔들어 보고 싶었습니다. `configmap-conn.yml` 을 열어 `conn_info` 의 포트만 80 에서 90 으로 살짝 고쳤습니다.

ex13/configmap-conn.yml (포트 변경)

```
# ... 생략

conn_info: "localhost:90"           # 환경변수 수정
```

```
kubectl apply -f ex13/configmap-conn.yml # 변경된 ConfigMap 적용
```

apply를 걸자 `configured` 메시지가 떴습니다. ConfigMap 자체는 갱신된 듯했는데, 막상 Pod 안의 환경 변수를 들여다보니 포트는 여전히 80 이었습니다.

'어? 분명히 바꿨는데 왜 그대로지?'

리눅스에서 환경 변수는 **프로세스가 시작될 때 한 번 꽂히는 값**입니다. 그래서 ConfigMap 쪽이 갱신되어도, 이미 떠 있는 Pod의 프로세스 안에는 처음 들어온 옛 값이 그대로 남아 있습니다. 새 값을 반영하려면 Pod를 한 번 새로 띄워 줘야 합니다.

× apply 만 한 경우 — Pod에 반영 안 됨



ConfigMap은 갱신됐지만, 이미 떠 있는 Pod의 환경 변수는 시작 시점에 박힌 값(80)을 그대로 가지고 있습니다.

그림 6-5. apply 만 한 경우 - ConfigMap 변경이 기존 Pod에는 반영되지 않음

✓ rollout restart 까지 한 경우 — 새 Pod에 반영 OK



새 Pod는 시작 시점에 갱신된 ConfigMap을 읽어 환경 변수에 새 값(90)을 박습니다.

그림 6-6. rollout restart 로 Pod를 새로 띄우면 ConfigMap 새 값이 환경 변수로 반영됨

```
kubectl rollout restart deployment nginx-config-secret # Pod 재시작
```

`kubectl rollout restart` 는 여러 Pod를 순차로 교체해 새 값을 반영하는 명령입니다. 재시작이 끝나고 다시 환경 변수를 찍어 보니 포트가 `90` 으로 바뀌어 있었습니다. `apply`는 ConfigMap을 갱신할 뿐, 실제 Pod까지 반영되려면 재시작이 한 번 더 필요합니다.

설정과 비밀번호를 이미지 바깥으로 빼는 일은 끝났습니다. 노트에 적어 둔 두 단어 중 첫 번째 매듭이 풀렸지만, 한 가지가 더 남아 있습니다. 비밀번호로 잠가 두는 DB의 데이터가 아직 그대로였습니다.

6.2 Volume - 데이터의 영속성 확보

6.2.1 Pod의 휘발성 문제

설정 분리는 끝났고, 남은 건 데이터였습니다. 챕터 4에서 본 그 자동 복구가 DB Pod에서는 다른 문제를 만듭니다. Pod는 기본적으로 **휘발성**이라, 안에서 만든 파일은 Pod 수명과 함께 사라집니다. 회원 가입과 게시글이 매일 쌓이는 DB가 이런 식으로 휘발되면 운영을 할 수가 없습니다.

'Pod 지워지면 DB 데이터도 같이 사라진다는 거구나.'

그리고 보니 챕터 2에서 본 Docker의 마운트와 같은 형태가 필요했습니다. 호스트나 별도 볼륨에 데이터를 빼두고 컨테이너는 그 경로를 끌어다 쓰는 방식입니다. Kubernetes에도 같은 기능이 있고, 이름이 **Volume**입니다.

****볼륨(Volume)****은 Pod 내부 컨테이너가 사용할 수 있는 외부 저장 공간입니다. Pod 수명과 분리되어 있어, Pod가 사라져도 데이터가 남을 수 있습니다.

Volume에는 여러 종류가 있습니다.

종류	설명	데이터 유지
<code>emptyDir</code>	Pod 생성 시 만들어지는 임시 저장 공간	Pod 삭제 시 함께 삭제
<code>hostPath</code>	워커 노드(호스트)의 특정 경로를 Pod에 마운트	노드에 남지만, Pod가 다른 노드로 이동하면 접근 불가
<code>PV / PVC</code>	클러스터가 관리하는 영구 저장소를 만들고 요청서(PVC)로 Pod에 연결	Pod가 삭제되어도 유지

쿠버네티스에서 데이터 보존을 위해 가장 보편적으로 사용하는 방식은 **PV(PersistentVolume)** 와 **PVC(PersistentVolumeClaim)** 입니다.

6.2.2 PV와 PVC

전체 실습 코드는 깃헙을 참고합니다.

실습 코드 (GitHub): <https://github.com/metacoding-10-linux-docker/docker/tree/master/ex14>

'Volume이 저장 공간이라는 건 알겠는데, PV랑 PVC는 왜 두 개로 나뉘져 있지?'

Pod가 직접 저장 공간을 관리하려고 들면 인프라 세부 사항까지 같이 짚어져야 합니다. 어느 디스크에, 어떤 권한으로, 몇 기가짜리를 쓸 것인지 매번 따져야 합니다. Kubernetes는 그래서 저장소를 **창고**와 그 창고를 쓰겠다는 **신청서**로 분리해 두었습니다. 한 장 그림으로 보면 관계가 분명해집니다.

Pod가 PVC를 거쳐 실제 저장소(PV)에 연결되는 구조



그림 6-7. PV는 실제 저장 공간, PVC는 그 공간을 요청하는 신청서

- ****PV(PersistentVolume)****는 실제 저장 공간, 즉 **창고**입니다. 용량, 권한, 위치 같은 창고의 사양이 정의됩니다.
- ****PVC(PersistentVolumeClaim)****는 창고를 쓰겠다고 작성하는 **신청서**입니다. **"1Gi짜리 읽기·쓰기 가능한 창고가 필요하다"**고 적어 두면, Kubernetes가 조건에 맞는 PV를 찾아 자동으로 PVC와 연결합니다.

Pod는 PV를 직접 건드리지 않고 PVC만 붙여 사용합니다. 실제 창고 위치는 PVC가 알아서 연결해 주기 때문에, Pod 입장에서는 **"용량이 맞는 저장 공간 하나"**만 보입니다.

PV 만들기

오픈이 ex14 폴더에서 PV YAML부터 펼쳤습니다.

```

# ex14/volume-pv.yml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: volume-pv
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteOnce
  storageClassName: "" # 자동 StorageClass 비활성, 아래 PVC에서 정적 바인딩
  hostPath:
    path: /mnt/data
    type: DirectoryOrCreate # 경로가 없으면 자동 생성
    
```

이번 실습에서는 외부 스토리지 없이 Minikube 내부의 경로(`/mnt/data`)를 저장소로 썼습니다.

`storageClassName: ""` 은 자동 발급을 비활성화하고, 지금 만든 이 PV에 정적으로 바인딩하게 하는 지정입니다.

StorageClass란

StorageClass는 PVC를 받으면 그 명세에 맞는 PV를 자동으로 만들어 주는 템플릿입니다. AWS·GCP·Minikube 같은 환경마다 기본 StorageClass가 따로 준비되어 있어 평소에는 PVC만 작성해도 PV가 자동 발급됩니다. 이번 실습은 직접 만든 PV에 묶어 두는 정적 바인딩을 보기 위해 빈 문자열로 비활성화했습니다.

PVC 만들기

```
# ex14/volume-pvc.yml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: volume-pvc
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: ""
  resources:
    requests:
      storage: 1Gi
  volumeName: volume-pv                # 바인딩할 PV 이름을 수동 지정
```

`volumeName`은 "이 PVC를 특정 PV에 수동으로 붙이라"는 지정입니다. 위 PV의 `metadata.name`과 같은 값을 적어 정적 바인딩을 보장합니다.

PVC와 PV가 연결되는 조건

PVC와 PV가 서로 연결되려면 세 가지가 맞아야 합니다. 첫째, **읽기·쓰기 방식**(accessModes)이 호환되어야 합니다. 둘째, **저장소 종류 이름**(storageClassName)이 같아야 합니다. 셋째, 신청한 **용량이 참고 용량보다 작거나 같아야** 합니다. 하나라도 어긋나면 PVC는 짝을 못 찾고 **Pending** 상태에 머무릅니다.

Pod에 연결

Pod 설정의 `volumes`에서 PVC를 선언하고, `volumeMounts`를 통해 컨테이너 내부 경로에 마운트합니다.

```
# ex14/volume-pod.yml
apiVersion: v1
kind: Pod
metadata:
  name: volume-pod
spec:
  containers:
  - name: nginx-volume
    image: nginx
    volumeMounts:
    - name: storage
      mountPath: /mnt/data
  volumes:
  - name: storage
    persistentVolumeClaim:
      claimName: volume-pvc
```

Pod 입장에서는 `/mnt/data` 폴더가 생긴 것과 같으며, 여기에 저장되는 데이터는 실제 PV가 가리키는 노드의 경로에 기록됩니다.

```
kubectl apply -f ex14/volume-pv.yml      # PV(참고) 생성
kubectl apply -f ex14/volume-pvc.yml     # PVC(참고 신청서) 생성
kubectl apply -f ex14/volume-pod.yml     # PVC를 마운트한 Pod 생성
kubectl get pv,pvc -o wide               # PV·PVC가 Bound 됐는지 확인
```

실행 결과

```
$ kubectl get pv,pvc -o wide
NAME CAPACITY ACCESS MODES RECLAIM POLICY STATUS CLAIM S...
persistentvolume/volume-pv 1Gi RWX Retain Bound default/volume-pvc <unset>
NAME STATUS VOLUME CAPACITY ACCESS MODES STORAGECLASS VOLUMEATTRIBUTESCL...
persistentvolumeclaim/volume-pvc Bound volume-pv 1Gi RWX <unset> 9s
```

그림 6-8. PV와 PVC가 Bound 상태로 연결된 결과

STATUS가 Bound 상태라면 PV와 PVC가 정상적으로 연결된 것입니다.

'참고와 신청서가 엮였구나.'

확인을 위해 Pod 내부에서 파일을 하나 만든 뒤, Pod를 삭제하고 다시 띄워 보겠습니다.

```
kubectl exec -it volume-pod -- /bin/bash      # Pod 안 bash 접속
touch /mnt/data/c.txt                          # 마운트 경로에 빈 파일 생성
exit                                           # Pod 셸 종료

kubectl delete pod volume-pod                  # 기존 Pod 삭제
kubectl apply -f ex14/volume-pod.yml           # 같은 PVC를 쓰는 Pod 재생성
kubectl exec -it volume-pod -- ls /mnt/data    # 새 Pod에서 c.txt가 남아 있는지 확인
```

실행 결과

```
$ kubectl exec -it volume-pod -- /bin/bash
root@volume-pod:/#
root@volume-pod:/# ls /mnt/data
c.txt
root@volume-pod:/#
```

그림 6-9. Pod가 새로 태어났는데도 c.txt가 그대로 남아 있는 모습

새로 띄운 Pod 안에서 `ls`를 쳤는데 `c.txt`가 그대로 보였습니다. Pod는 통째로 한 번 사라졌다가 새로 태어났는데, 파일은 PV 안에 머물러 있었습니다. PVC가 새 Pod에도 똑같은 참고를 다시 연결해 준 결과입니다.

'Pod가 새로 만들어져도 참고는 그대로구나. 이제 데이터가 사라질 걱정이 없겠다.'

회의실 화이트보드에 적어 둔 두 단어, **설정·비밀번호**와 **데이터** 위에 줄이 그어졌습니다.

6.3 통합 실습 - 쿠버네티스 위에 웹사이트 올리기

전체 실습 코드는 깃헙을 참고합니다.

실습 코드 (GitHub): <https://github.com/metacoding-10-linux-docker/docker/tree/master/ex15>

다음 날 아침, 오픈이는 자리에 앉자마자 노트를 펼쳤습니다. 며칠 전 회의실에서 팀장이 의뢰한 챕터 3 그 사이트를 K8s 위에 올릴 차례였습니다. 매듭은 풀렸고, 도구는 쥐고 있었습니다. 이번 챕터에서는 그동안 따로따로 익힌 도구들이 한 사이트에서 한꺼번에 만납니다. 챕터 4·5에서 본 Pod·Deployment·Service·Ingress에, 이번 챕터에서 새로 더한 ConfigMap·Secret과 PV·PVC가 합쳐집니다.

'지금까지는 따로따로 띄워 본 건데, 이것들이 한꺼번에 올라가도 진짜 설계도처럼 맞물려 돌아갈까?'

부분 실습은 잘 됐습니다. 그런데 전체 시스템이 한 덩어리로 살아 움직이는 모습은 아직 머릿속에 잘 그려지지 않았습니다. 화면을 응시하던 오픈이의 등 뒤로 팀장의 목소리가 들려왔습니다.

팀장 : "부분 실습은 끝났죠. 한 번에 연결해 봐야 진짜 실력이 늘어요."

이번 절은 단순한 실습의 연장이 아니었습니다. 챕터 3에서 docker-compose로 띄워 봤던 그 사이트를 이번에는 클러스터 위에 그대로 다시 세우는 자리였습니다.

6.3.1 전체 그림

오픈이가 펼쳐 본 구성도에는 서비스 네 개가 들어 있었습니다. 프론트엔드(Nginx), 백엔드(Spring Boot), DB(MySQL), 그리고 방문 횟수를 기록할 Redis. 챕터 3에서 본 세 덩어리에 Redis가 더해져, 운영 환경에 한층 더 가까운 구조였습니다.

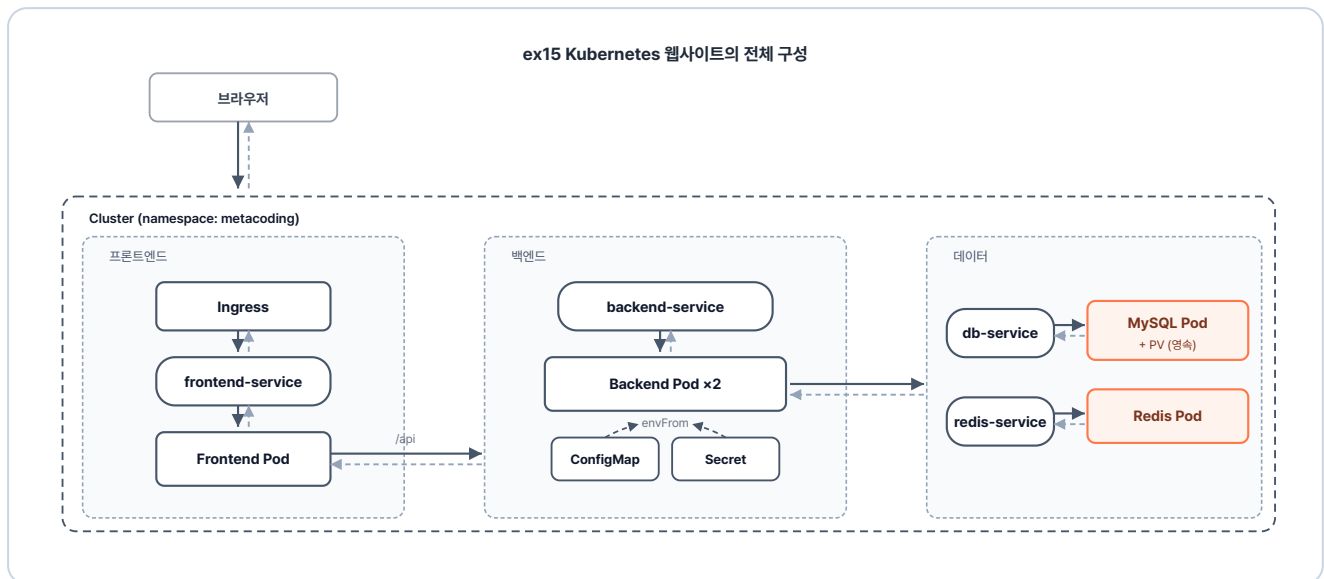


그림 6-10. ex15 Kubernetes 웹사이트의 전체 구성

오픈이는 그림을 따라가며 흐름을 머릿속에 그려 봤습니다. 브라우저의 요청이 Ingress를 통과해 Frontend Service를 거쳐 프론트엔드 Pod에 닿습니다. 사용자가 게시판이나 로그인 버튼을 누르면 프론트엔드의 Nginx가 `/api/ ...` 요청을 받아 클러스터 내부의 Backend Service로 넘겨줍니다. 요청을 받은 백엔드 Pod는 로직을 처리하는 과정에서 DB Service와 Redis Service를 호출하여 데이터를 읽고 씁니다.

그림을 보고 있자니 한 가지가 분명해졌습니다. Pod 사이의 모든 통신이 IP가 아닌 서비스 이름으로 이루어진다는 사실이었습니다. 챕터 5에서 본 클러스터 DNS, 즉 **CoreDNS**가 서비스 이름을 ClusterIP로 변환해 주기 때문에, 복잡한 숫자 주소 대신 `db-service`, `backend-service` 같은 직관적인 이름만

챙기면 됩니다.

'IP가 어디로 옮겨 다니든 이름만 잡고 있으면 흐름이 안 끊기는구나.'

6.3.2 폴더 구조와 진행 방식

오픈이가 ex15 폴더를 열어 보니 이미지를 만드는 영역과 쿠버네티스 설정(YAML) 영역이 깔끔하게 나뉘어 있었습니다. 이번 실습의 초점은 이미지 제작이 아니라 쿠버네티스 리소스들이 어떻게 한 덩어리로 맞물리는지 보는 데 있었습니다.

```

ex15/
├── backend/
│   ├── Dockerfile
│   └── entrypoint.sh
├── db/
│   ├── Dockerfile
│   └── init.sql
├── frontend/
│   ├── Dockerfile
│   ├── index.html
│   └── nginx.conf
├── redis/
│   └── Dockerfile
├── k8s/
│   ├── namespace.yml
│   ├── backend/
│   │   ├── backend-configmap.yml
│   │   ├── backend-deploy.yml
│   │   ├── backend-secret.yml
│   │   └── backend-service.yml
│   ├── db/
│   │   ├── db-deploy.yml
│   │   ├── db-pv.yml
│   │   ├── db-pvc.yml
│   │   ├── db-secret.yml
│   │   └── db-service.yml
│   ├── frontend/
│   │   ├── frontend-deploy.yml
│   │   ├── frontend-ingress.yml
│   │   └── frontend-service.yml
│   └── redis/
│       ├── redis-deploy.yml
│       └── redis-service.yml
└── README.md

```

```

# Spring Boot 백엔드 이미지
# JDK 이미지 + entrypoint.sh 복사
# Git clone + Gradle 빌드 + JAR 실행
# MySQL 이미지
# MySQL 이미지 + init.sql 복사
# 테이블·초기 데이터 생성 스크립트
# NGINX + HTML 이미지
# nginx 이미지 + index.html·nginx.conf 복사
# 로그인/게시판 UI (방문 카운터 표시)
# /api 경로를 backend-service로 프록시
# Redis 이미지
# redis 공식 이미지 기반
# 쿠버네티스 리소스 매니페스트
# ex15 네임스페이스 정의

# 비밀이 아닌 설정값
# 백엔드 Deployment
# DB 비밀번호 등 민감 정보
# 내부용 ClusterIP Service

# MySQL Deployment
# PersistentVolume (노드 로컬 저장소)
# PersistentVolumeClaim (볼륨 요청)
# MySQL 계정 정보
# 내부용 ClusterIP Service

# 프론트 Deployment
# 외부 진입점 (Ingress)
# 내부용 ClusterIP Service

# Redis Deployment
# 내부용 ClusterIP Service
# 실습 안내

```

6.3.3 리소스 살펴보기

오픈이는 `ex15/k8s/` 폴더를 열고 그 안의 리소스들을 하나씩 훑어 봤습니다. 서비스별로 폴더가 나뉘어 있었고, 각 폴더에는 Deployment와 Service가 기본으로 들어 있었습니다. 이번 절에서 오픈이가 보고 싶었던 건 이들이 한꺼번에 실행될 때 어떻게 서로 맞물려 돌아가는지였습니다.

Namespace - 논리적 분리

지금까지 만든 리소스는 모두 default라는 기본 공간에 담겨 있었습니다. 그래서 `kubectl get` 을 칠 때마다 이전 실습의 잔해까지 함께 쏟아져 나와 화면이 어수선했습니다.

'혼자 연습할 때는 괜찮지만, 여러 팀의 서비스가 같이 돌면 이름이 충돌할 수도 있겠구나.'

한 회사 안에서도 팀마다 사무실을 따로 쓰듯, 쿠버네티스에서도 공간을 분리할 수 있습니다. Namespace는 하나의 클러스터를 논리적으로 구분해 주는 가상 공간입니다.

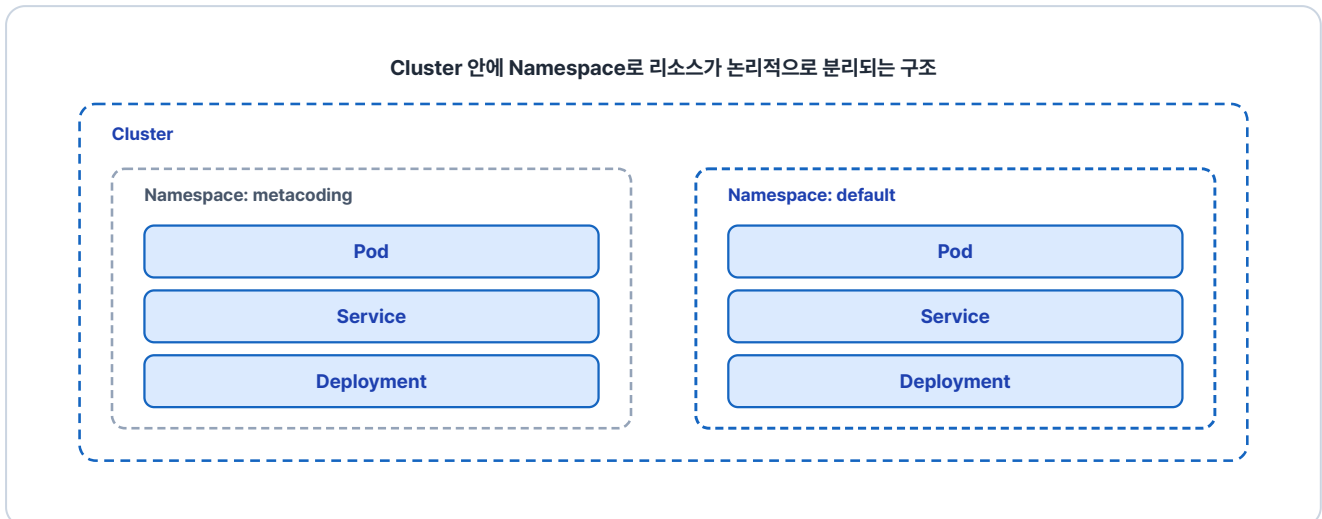


그림 6-11. 같은 클러스터 안에서 Namespace가 리소스를 층처럼 분리

Namespace는 리소스를 논리적으로 구분하는 가상 공간입니다. 별도 지정이 없으면 모든 리소스는 **default** 네임스페이스에 소속됩니다.

이번 실습에서는 `metacoding` 이라는 전용 네임스페이스를 만들어 모든 리소스를 그 안에서 관리하기로 했습니다.


```
# ex15/k8s/namespace.yml
apiVersion: v1
kind: Namespace
metadata:
  name: metacoding
```

이후 만들어지는 모든 리소스는 metacoding이라는 독립된 영역에 자리를 잡습니다.

① Frontend

오픈이는 frontend 폴더부터 펼쳤습니다. 외부에서 들어오는 모든 요청이 처음 닿는 입구를 정의하는 폴더였습니다. 핵심은 단일 path로 모든 요청을 잡아 frontend-service로 넘기는 `frontend-ingress.yml` 입니다.

ex15/k8s/frontend/frontend-ingress.yml

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: frontend-ingress
  namespace: metacoding
spec:
  ingressClassName: nginx      # 어느 Controller가 이 규칙을 집행할지 지정
  rules:
    - http:
        paths:
          - path: /              # 모든 경로를 잡음
            pathType: Prefix
            backend:
              service:
                name: frontend-service
                port:
                  number: 80
```

`path: /` + `pathType: Prefix` 는 들어오는 모든 요청을 frontend-service의 80번 포트로 넘기겠다는 규칙입니다. 정적 콘텐츠는 프론트엔드가 그대로 응답하고, `/api/...` 요청은 프론트엔드의 Nginx 설정에 따라 클러스터 내부의 backend-service로 다시 흘러갑니다.

같은 폴더의 나머지 두 파일은 다음과 같습니다.

파일	역할
<code>frontend-deploy.yml</code>	<code>metacoding/frontend:1</code> 이미지를 띄우는 Deployment. 정적 HTML과 <code>/api</code> 를 backend-service로 프록시하는 Nginx 설정을 담은 컨테이너 한 개를 80번 포트로 올립니다
<code>frontend-service.yml</code>	위 Deployment의 Pod를 라벨로 묶는 ClusterIP Service. 80번 포트를 클러스터 안에서만 노출해 Ingress가 이 이름으로 찾아갑니다

② Backend

backend 폴더에서 가장 손이 많이 간 매니페스트는 `backend-deploy.yml`이었습니다. 챕터 4에서 본 `replicas`와 6.1에서 따로따로 다룬 ConfigMap·Secret이 한 매니페스트에서 만나는 자리였습니다.

ex15/k8s/backend/backend-deploy.yml

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend-deploy
  namespace: metacoding          # 모든 리소스를 metacoding 네임스페이스에 배치
spec:
  replicas: 2                    # Pod 두 개로 트래픽 분산
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      containers:
        - name: backend-server
          image: metacoding/backend:1
          ports:
            - containerPort: 8080    # Tomcat 기본 포트
          envFrom:
            - configMapRef:
                name: backend-configmap    # DB 주소·JDBC URL·Redis 호스트 등 일반 설정
            - secretRef:
                name: backend-secret      # DB 계정·비밀번호

```

`envFrom` 아래에 ConfigMap과 Secret을 한꺼번에 매달면 두 리소스의 모든 키가 컨테이너 환경 변수로 그대로 들어옵니다. ConfigMap의 호스트 자리에는 IP가 아니라 `db-service`·`redis-service` 같은 Service 이름이 적혀 있어, DB Pod가 죽었다 살아나도 ConfigMap을 손댈 필요가 없습니다.

같은 폴더의 다른 세 파일은 다음과 같습니다.

파일	역할
<code>backend-service.yml</code>	두 백엔드 Pod 앞에 서는 ClusterIP Service. 8080 포트를 <code>backend-service</code> 이름으로 묶어 프론트엔드와 클러스터 안 다른 Pod가 호출합니다
<code>backend-configmap.yml</code>	DB 접속 URL, JDBC 드라이버 클래스, Redis 호스트·포트처럼 환경마다 달라지지만 비밀이 아닌 값들을 담습니다
<code>backend-secret.yml</code>	DB 사용자명·비밀번호 같은 민감 정보를 따로 분리해 보관합니다

③ DB

db 폴더의 `db-deploy.yml` 은 6.2에서 단일 Pod에 붙여 본 PVC를 Deployment 한가운데로 끌어들이는 형태였습니다.

ex15/k8s/db/db-deploy.yml

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: db-deploy
  namespace: metacoding
spec:
  replicas: 1 # DB는 단일 인스턴스로 운영
  selector:
    matchLabels:
      app: db
  template:
    metadata:
      labels:
        app: db
    spec:
      containers:
        - name: db-server
          image: metacoding/db:1
          ports:
            - containerPort: 3306 # MySQL 기본 포트
          envFrom:
            - secretRef:
                name: db-secret # MySQL 계정·비밀번호
          volumeMounts:
            - name: data
              mountPath: /var/lib/mysql # MySQL이 데이터를 쓰는 경로를 PV로
      volumes:
        - name: data
          persistentVolumeClaim:
            claimName: db-pvc

```

`volumeMounts`로 컨테이너의 `/var/lib/mysql` (MySQL이 데이터를 쓰는 표준 경로)에 PVC가 가리키는 PV를 끌어다 붙입니다. Pod가 사라져도 데이터는 PV에 남아, Deployment가 새 Pod를 띄워도 회원 정보와 게시글이 같은 자리에서 다시 보입니다.

같은 폴더의 나머지 네 파일은 다음과 같습니다.

파일	역할
<code>db-service.yml</code>	DB Pod 앞에 서는 ClusterIP Service. 3306 포트를 <code>db-service</code> 이름으로 묶어 백엔드가 ConfigMap의 호스트 값으로 호출합니다
<code>db-secret.yml</code>	MySQL 루트 비밀번호와 앱 사용자 계정 정보를 담습니다. backend-secret과는 별개로 DB 컨테이너 자체의 초기화에 쓰입니다
<code>db-pv.yml</code>	노드의 <code>/data/mysql</code> 경로를 가리키는 1Gi 크기 PersistentVolume
<code>db-pvc.yml</code>	1Gi짜리 PVC. <code>volumeName</code> 으로 위 db-pv를 정적으로 바인딩해 db-deploy의 <code>volumes</code> 가 이 PVC를 통해 PV로 연결됩니다

④ Redis

redis 폴더는 한층 단출했습니다. 백엔드가 방문 횟수를 기록하기 위해 호출하는 인메모리 저장소이므로 별도 ConfigMap이나 PVC 없이 두 파일이면 충분했습니다.

파일	역할
<code>redis-deploy.yml</code>	<code>metacoding/redis:1</code> 이미지를 띄우는 Deployment. Redis 컨테이너 한 개를 6379 포트로 올립니다
<code>redis-service.yml</code>	<code>redis-service</code> 이름으로 6379 포트를 노출하는 ClusterIP Service. 백엔드의 ConfigMap이 이 이름을 호스트 값으로 참조합니다

6.3.4 공통 준비

코드를 살펴봤으니 띄울 차례였습니다. 오픈이는 본격적으로 배포하기 전에 두 가지부터 챙겼습니다. Minikube와 Ingress Controller를 켜는 일, 그리고 네 가지 이미지를 빌드해 두는 일이었습니다.

```
minikube start
minikube addons enable ingress
kubectl get pod -n ingress-nginx # Controller Pod Running 확인
```

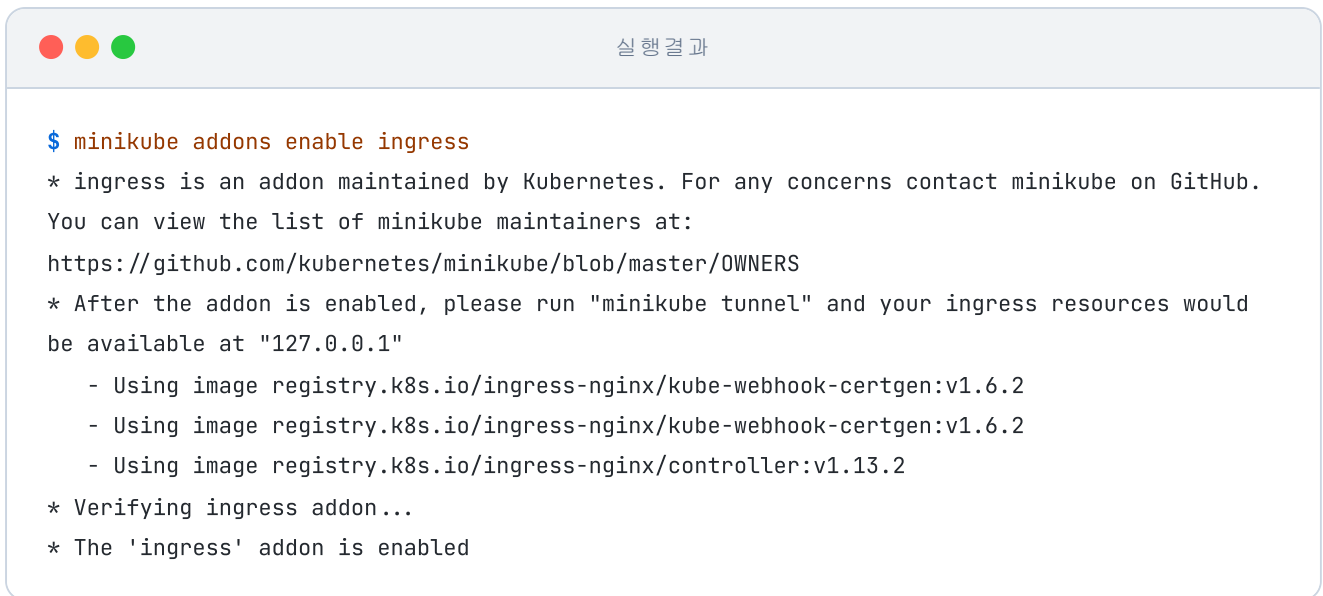


그림 6-12. Nginx Ingress Controller 애드온 설치

Minikube는 독립적인 가상 환경에서 작동하는 클러스터라, 로컬 호스트에서 빌드한 이미지를 곧장 인식하지 못합니다. 그래서 별도의 레지스트리를 두지 않고 `minikube image build` 명령으로 Minikube 자체에 이미지를 직접 만들어 두기로 했습니다.

```
minikube image build -t metacoding/db:1 ex15/db # DB 이미지 빌드
minikube image build -t metacoding/backend:1 ex15/backend # 백엔드 이미지 빌드
minikube image build -t metacoding/frontend:1 ex15/frontend # 프론트엔드 이미지 빌드
minikube image build -t metacoding/redis:1 ex15/redis # Redis 이미지 빌드
```

6.3.5 한 번에 배포하고 결과 확인

리소스 구경을 마친 오픈이는 `ex15/k8s/` 폴더를 통째로 넘겼습니다.

```
kubectl apply -f ex15/k8s/namespace.yml # Namespace 먼저
kubectl apply -f ex15/k8s/ --recursive # k8s 이하 폴더의 리소스 전부 일괄 배포
```

명령을 치자 모든 리소스가 한꺼번에 생성됐습니다. 오픈이는 상태를 보기 위해 곧바로 조회 명령을 입력했습니다.

```
kubectl get deploy,pod,service,ingress -n metacoding # 네임스페이스 리소스 상태 일괄 조회
```

프론트엔드와 데이터베이스는 금세 올라왔습니다. 백엔드 Pod만 빌드와 의존성 설치 과정 때문에 ContainerCreating 상태에 한참 머물러 있었습니다.

'다른 건 다 댔는데 백엔드만 늦네.'

로그를 들여다보니 안에서 Gradle 빌드가 한창이었습니다. 잠시 기다리자 모든 Pod가 Running 상태로 바뀌었고, 오픈이는 외부 접근을 위한 통로를 열었습니다. 챕터 5에서처럼 별도 터미널에서 `minikube tunnel` 을 실행하면 Ingress 입구가 `localhost` 로 이어집니다.

```
minikube tunnel
```

```
# 새 터미널에서 실행 (관리자 권한 요구)
```

브라우저에서 `http://localhost` 로 접속해 결과를 확인했습니다.

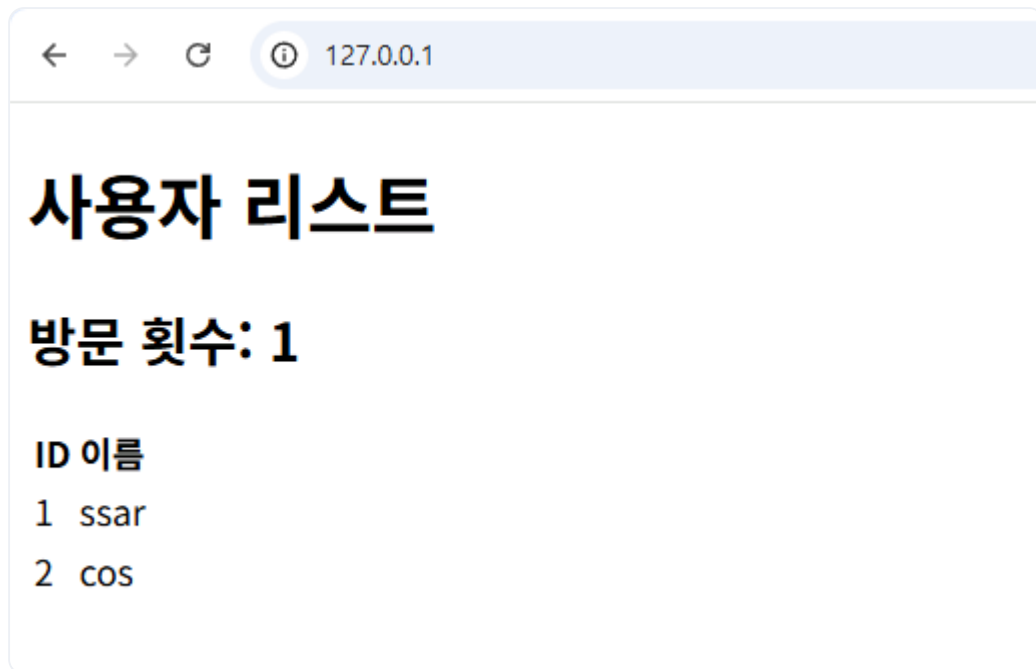


그림 6-13. Ingress를 거쳐 웹사이트가 화면에 응답

화면에는 데이터베이스에서 불러온 정보와 함께 방문 횟수가 떠 있었습니다. 새로고침을 할 때마다 숫자가 한 칸씩 올라갔습니다. 네 개의 서비스가 서로 이름을 부르며 제대로 맞물려 돌아가고 있다는 증거였습니다.

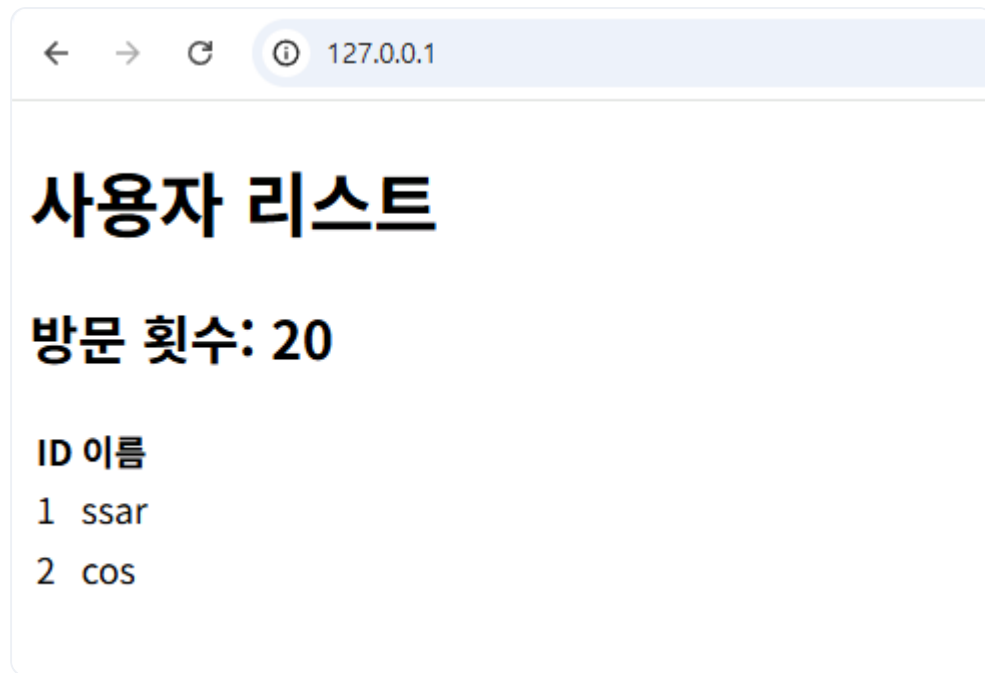


그림 6-14. 새로고침 시 방문 횟수가 증가

백엔드 두 Pod에 요청이 실제로 분산되는지는 로그로도 확인할 수 있었습니다.

```
kubectl logs -l app=backend -n metacoding --tail=100 --prefix # backend Pod 최근 100 줄 (Pod명 표시/)
```

서로 다른 Pod 이름이 앞에 붙은 로그 줄이 번갈아 나타난다면 요청이 두 서버에 분산되어 들어간 것입니다.

옆자리 동료가 의자를 굴러 화면을 들여다봤습니다.

동료 : "어, 이거 챕터 3에서 docker-compose로 띄웠던 그 사이트 아니에요? 이번엔 K8s 위에 있네요."

오픈이는 고개를 끄덕였습니다. 같은 사이트지만 자리가 달라졌습니다. docker-compose 한 파일 안에 묶여 있던 세 덩어리가, 이제는 클러스터 위에서 각자 Pod로 떠 있고 Service 이름으로 서로를 호출하고 있었습니다.

'드디어 다 엮였다. 이름만으로 서로를 찾아가는 통합 시스템이라니.'

이것만은 기억하자

- **설정과 비밀번호는 이미지 바깥에 둡니다.** ConfigMap에는 환경마다 달라지는 일반 설정값을, Secret에는 비밀번호·토큰 같은 민감 정보를 두고 `envFrom`으로 한꺼번에 주입합니다.
- **ConfigMap을 바꾼 뒤에는 Pod를 새로 띄웁니다.** 환경 변수는 프로세스 시작 시점에 한 번 꽂히는 값이라 `apply` 만으로는 갱신이 반영되지 않습니다. `kubectl rollout restart`로 Pod를 교체해야 새 값이 박힙니다.
- **데이터는 PV·PVC로 Pod 바깥에 두어 보존합니다.** Pod는 휘발성이라 안에서 만든 파일은 함께 사라지지만, PVC가 가리키는 PV에 데이터를 두면 Pod가 새로 태어나도 그대로 남습니다.
- **이름 하나로 한 시스템이 됩니다.** `kubectl apply -f` 한 줄로 Namespace·Deployment·Service·Ingress·ConfigMap·Secret·PV·PVC가 동시에 올라가도, Service 이름만 맞으면 CoreDNS가 ClusterIP를 풀어 통합 서비스가 완성됩니다.

마치며

퇴근길 지하철에서 오픈이는 노트를 펼쳤다 다시 덮었습니다. 며칠 전 회의실 화이트보드에 적어 두었던 두 단어, **설정·비밀번호**와 **데이터** 위로 줄이 두 개 그어져 있었습니다. 그 옆에는 챕터 3의 통합 사이트가 클러스터 위에서 새로 도는 모습이 작게 그려져 있었습니다.

여기까지가 이 책에서 짚은 흐름입니다. 한 명의 개발자가 하나의 클러스터 위에 하나의 서비스를 올리는 자리. 그 너머의 운영은 이 책의 범위 밖이지만, 다음에 공부할 거리를 제목만 짚어 두면 덜 막막합니다.

- **StatefulSet:** 상태를 가진 Pod (DB 클러스터 등)
- **DaemonSet:** 모든 노드에 배포되는 Pod (로그 수집 등)
- **Job / CronJob:** 일회성·정기 작업
- **HPA(HorizontalPodAutoscaler):** 자동 스케일링
- **RBAC:** 역할 기반 접근 제어
- **NetworkPolicy:** Pod 간 통신 제한
- **Helm:** 패키지 관리

처음부터 모든 걸 알고 시작할 수는 없습니다. 챕터 1에서 선배가 던진 "**환경이 달라서 그래요. Docker 한번 알아봐요.**" 한 마디가 여기까지 온 출발점이었습니다.

다음에 새 장애를 만나거나 새 리소스를 마주치면, 이 책에서 익힌 흐름을 그대로 꺼내 쓰면 됩니다. 컨테이너 한 칸에서 시작해 클러스터 전체를 손에 잡기까지의 거리는, 이미 한 번 걸어본 길이 됐습니다. 내일도 같은 시간에 출근입니다. 또 한 번 새로운 문제가 책상 위에 놓일 것입니다.

이번에는 그 문제 앞에서 덜 막막할 것입니다.